

# CURSO DE PYTHON

**Corporación Ecuatoriana para el Desarrollo de la  
Investigación y la Academia**

**Ing. María Fernanda Granda J., PhD**

**16-07-2026**

**“Conectando ideas,  
transformando  
sociedades”**

## **Tema 2: Programación Avanzada en Python**

- Introducción y motivación
- Iteradores
- Generadores
- Decoradores
- Modularización
- Buenas prácticas (PEP 8)
- Taller integrador
- Tarea

## **Tema 2: Programación Avanzada en Python**

### **Objetivo de Aprendizaje:**

Al finalizar la sesión, el estudiante será capaz de optimizar programas mediante el uso de iteradores, generadores y decoradores, organizar proyectos en módulos reutilizables y aplicar buenas prácticas de desarrollo siguiendo estándares profesionales.

### **Competencias:**

- Comprender el funcionamiento interno de la iteración en Python.
- Crear generadores para optimizar memoria.
- Implementar decoradores para reutilizar lógica.
- Organizar proyectos mediante módulos y paquetes.
- Aplicar principios básicos de calidad de código y PEP 8.

## ¿Por qué necesitamos programación avanzada?

### Problema inicial:

- En ejercicios anteriores trabajamos con listas pequeñas:

```
numeros = [1,2,3,4,5]  
cuadrados = [x**2 for x in numeros]
```

- Esto funciona perfectamente porque solo existen cinco elementos.
- Ahora imaginemos un escenario diferente.

```
numeros = range(1000000)  
  
cuadrados = [x**2 for x in numeros]
```

## ¿Por qué necesitamos programación avanzada?

**Analizando el proceso:** Python hace lo siguiente:

- Recorre todos los números.
- Calcula cada cuadrado.
- Crea una lista.
- Guarda los 1 000 000 resultados en memoria RAM.

Es decir, antes de que podamos utilizar el primer resultado, Python ya terminó de calcularlos todos.

Visualmente sería:

```
1  + 1
2  + 4
3  + 9
...
999999 + 999998000001
```

Todo queda almacenado en memoria

## ¿Por qué necesitamos programación avanzada?

### Problema 1: Uso de Memoria

- Cada número ocupa espacio.
- Una lista con un millón de elementos puede consumir decenas de megabytes.
- ¿Qué pasaría si el procesamiento fuera:

```
10 millones  
100 millones  
1000 millones
```

- la memoria podría agotarse.

```
RAM disponible
```



```
Lista enorme
```

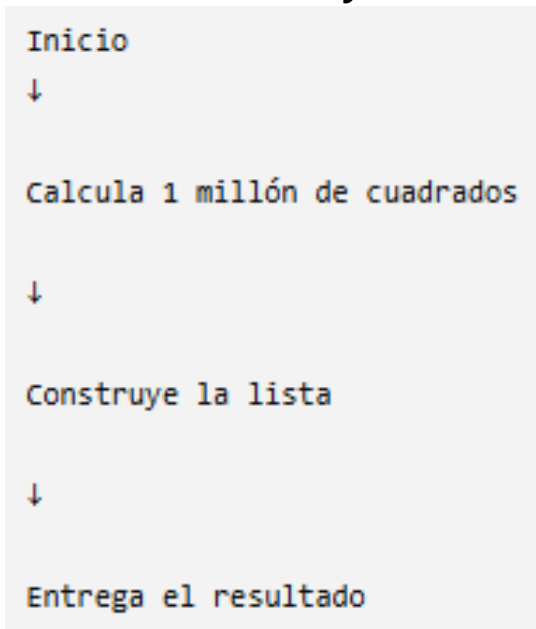


- Cuando la memoria se llena aparecen problemas como.
  - Lentitud
  - intercambio con disco (swap)

## ¿Por qué necesitamos programación avanzada?

### **Problema 2:** Tiempo de procesamiento

- Antes de mostrar el primer resultado, Python debe calcular todos.



- Si solo necesitábamos los primeros cinco valores, todo ese trabajo fue innecesario.

## ¿Por qué necesitamos programación avanzada?

### **Problema 3:** Escalabilidad

Supongamos que ahora trabajamos con:

- sensores IoT
- registros bancarios
- logs de servidores
- datos satelitales
- millones de imágenes

Ya no podemos cargar todo en memoria.

Necesitamos procesar los datos uno por uno.

## ¿Por qué necesitamos programación avanzada?

Surge una pregunta muy importante.

## ¿Existe una forma de generar los datos únicamente cuando los necesitemos?

La respuesta es:

**Sí.**

Y ahí aparecen los **iteradores y generadores**.

## Iteradores

### Concepto:

- Un iterador entrega un elemento a la vez.

En lugar de decir:

Aquí tienes un millón de resultados.

Dice:

Aquí tienes el primero.

¿Quieres otro?

Aquí tienes el segundo.

¿Otro?

Aquí está el tercero...

### Visualmente:

Lista

[1][4][9][16][25][36]...

↑

Todo almacenado

Iterador

1

↓

4

↓

9

↓

16

↓

25

## Iteradores

### Concepto:

- En Python, un iterador es un objeto que puede recorrer elementos uno por uno.
- En lugar de crear la lista completa, van calculando cada resultado cuando es solicitado.

### Ejemplo sencillo:

#### Salida:

10

20

30

```
lista = [10, 20, 30]

iterador = iter(lista)

print(next(iterador))
print(next(iterador))
print(next(iterador))
```

## Iteradores

¿Qué ocurre internamente en un for?

```
for numero in lista:  
    print(numero)
```

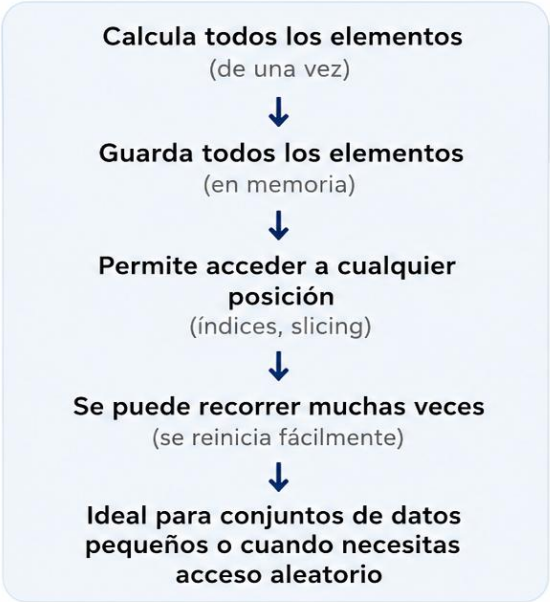
Python ejecuta algo similar a

```
iterador = iter(lista)  
  
while True:  
    try:  
        elemento = next(iterador)  
        print(elemento)  
    except StopIteration:  
        break
```

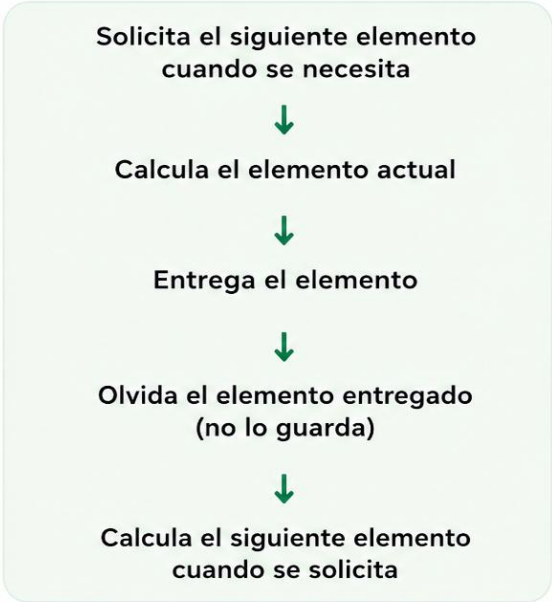
# Iteradores

## Comparación

### Lista



### Iterador



|                   |                                                         |                                                                             |
|-------------------|---------------------------------------------------------|-----------------------------------------------------------------------------|
| MEMORIA           | Alta (almacena todos los elementos)                     | Muy baja (solo mantiene el estado actual)                                   |
| ACCESO            | Aleatorio (por índice)                                  | Secuencial (solo siguiente)                                                 |
| VELOCIDAD INICIAL | Más lenta (calcula todo al inicio)                      | Más rápida (empieza a producir de inmediato)                                |
| USOS TÍPICOS      | Listas pequeñas, acceso frecuente, consultas aleatorias | Archivos grandes, streams de datos, procesamiento en línea, datos infinitos |

## ¿Cuándo usar un Iterador?

Cuando necesitas controlar completamente el comportamiento del recorrido.

- Decidir cómo avanzar, retroceder o detenerse

Por ejemplo:

- recorrer una estructura de datos propia;
- leer un archivo de una forma especial;
- recorrer un árbol binario;
- recorrer un grafo;
- implementar un algoritmo complejo.

## Iteradores

### ¿Dónde se utilizan?

Los generadores aparecen constantemente en aplicaciones reales.

Por ejemplo:

- leer archivos enormes

```
for linea in archivo:
```

- Python no carga el archivo completo.
- Lee una línea.
- Luego otra.
- Después otra.

## Iteradores

### ¿Dónde se utilizan?

- Procesamiento de logs

Servidor



100 millones de registros



Se procesan uno por uno

## Iteradores

### ¿Dónde se utilizan?

- Bases de datos

```
SELECT *
```

↓

Los registros llegan poco a poco

## Iteradores

### ¿Dónde se utilizan?

APIs

- Muchas APIs devuelven datos paginados.
- No descargan toda la información de una vez.

## Iteradores

### Crear un iterador propio

```
class Contador:

    def __init__(self, limite):
        self.limite = limite
        self.numero = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.numero < self.limite:
            self.numero += 1
            return self.numero

        raise StopIteration
```

**Uso:** Python realiza tres acciones:

- Crea un objeto nuevo de la clase Contador.
- Llama automáticamente al método `__init__`.

```
contador = Contador(5)

for x in contador:
    print(x)
```

```
print(next(contador))
print(next(contador))
print(next(contador))
```

- Pasa el objeto recién creado como `self` y el valor 5 como `limite`.

## Actividad 1

Crear un iterador personalizado

Desarrollar:

```
class TablaMultiplicar:
```

Que genere:

```
5  
10  
15  
20  
25  
...
```

Hasta un límite indicado

## Entregable

- Subir al aula virtual de CEDIA hasta el lunes 20 de Julio 6:59 pm
- Archivo .py
- Código documentado

## Generadores

### Motivación

Problema:

```
numeros = [x for x in range(1000000)]
```

Consume mucha memoria.

Solución: Generadores

```
def numeros():  
    yield 1  
    yield 2  
    yield 3
```

Uso:

```
for n in numeros():  
    print(n)
```

## Diferencia entre return y yield

- **Return**

```
def ejemplo():  
    return 1
```

Termina la función

- **Yield**

```
def ejemplo():  
    yield 1  
    yield 2  
    yield 3
```

Mantiene el estado

- La principal diferencia es que **return** finaliza la ejecución de una función y devuelve un único valor.
- En contraste, **yield** pausa la función temporalmente, devolviendo un valor y permitiendo que se reanude exactamente donde se quedó cuando se le vuelva a llamar.

## Ejemplo práctico

Ejemplo práctico:

Puedes detener el generador por cantidad de términos:

1. Entra al generador.
2. Ejecuta el código hasta encontrar yield.
3. Devuelve ese valor.
4. Guarda el estado del generador.
5. La siguiente llamada continúa desde donde quedó.

```
# definición
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

#uso
fib = fibonacci()

for _ in range(10):
    print(next(fib))
```

| Llamada   | Valor devuelto | Estado interno después |
|-----------|----------------|------------------------|
| next(fib) | 0              | a=1 , b=1              |
| next(fib) | 1              | a=1 , b=2              |
| next(fib) | 1              | a=2 , b=3              |
| next(fib) | 2              | a=3 , b=5              |
| next(fib) | 3              | a=5 , b=8              |

Todo generador es un iterador, pero no todo iterador es un generador.

## Iterador



## Generador



|               |                                                                 |                                                                    |
|---------------|-----------------------------------------------------------------|--------------------------------------------------------------------|
| ORIGEN        | Recorre una colección existente                                 | Produce valores dinámicamente                                      |
| CÓMO SE CREA  | llamando a iter(objeto) o implementando __iter__() y __next__() | Con una función que usa yield (se convierte en iterador)           |
| MEMORIA       | Mayor (ya que la colección existe completa en memoria)          | Muy baja (solo mantiene el estado actual)                          |
| RECORRIDO     | Secuencial, pero la colección permite acceso aleatorio          | Secuencial únicamente                                              |
| REUTILIZACIÓN | Se puede recorrer muchas veces                                  | Generalmente se recorre una sola vez (se agota)                    |
| EJEMPLOS      | iter([1, 2, 3]), iter("hola"), iter(diccionario)                | funciones con yield, expresiones generadoras (x for x in range(5)) |
| CUÁNDO USAR   | Cuando ya tienes todos los datos y necesitas recorrerlos        | Cuando los datos son grandes, infinitos o costosos de generar      |



**Resumen:** Todo generador es un iterador, pero no todo iterador es un generador. Los generadores son una forma más sencilla y eficiente de crear iteradores.



Eficiencia



Rendimiento



Escalabilidad

## Actividad 2

Construir un generador que produzca:

2  
4  
8  
16  
32  
64  
...

Potencias de 2.

## Entregable

- Subir al aula virtual de CEDIA hasta el lunes 20 de Julio 6:59 pm
- Archivo .py
- Código documentado

## Comparación de Memoria

Comparación de memoria

```
import sys

lista = [x for x in range(100000)]

gen = (x for x in range(100000))

print(sys.getsizeof(lista))
print(sys.getsizeof(gen))
```

Analizar resultados

Nota: **sys.getsizeof()** no mide toda la memoria consumida por los datos internos, sino únicamente el tamaño del objeto principal

## Comparación de Memoria

Los resultados pueden variar dependiendo de la versión de Python y SO.

800984

200

¿Por qué ocurre esto?

```
lista
|
├─ puntero → entero 0
├─ puntero → entero 1
├─ puntero → entero 2
├─ puntero → entero 3
└─ ...
```

La lista necesita memoria para:

- almacenar los 100 000 elementos;
- almacenar referencias a esos elementos;
- administrar la propia lista.

Por eso ocupa cientos de miles de bytes.

```
gen
|
├─ posición actual
├─ variables locales
├─ estado del yield
└─ referencia al rango
```

No almacena los números

Sólo recuerda:

- dónde quedó
- qué función está ejecutando
- cuáles son sus variables

Por eso ocupa alrededor de 200 bytes

## Comparación de Memoria

Después de crear el generador:

Ejecutar:

```
print(next(gen))  
print(next(gen))  
print(next(gen))
```

Luego vuelve a medir:

```
print(sys.getsizeof(gen))
```

Verás que el tamaño prácticamente no cambia.

### ¿Por qué?

Porque el generador no guarda los números generados, solo mantiene el estado necesario para continuar desde el siguiente valor.

## Comparación del Generador con la Lista

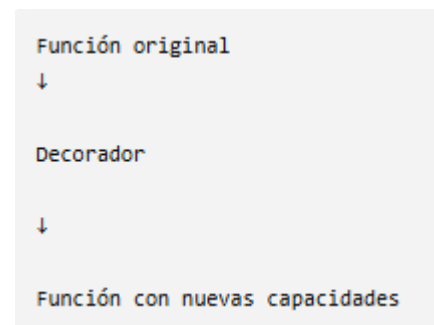
### Comparación

| Lista                                                                | Generador                                                                                                                  |
|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Calcula todos los elementos inmediatamente.                          | Calcula un elemento cuando se necesita.                                                                                    |
| Guarda todos los elementos en memoria.                               | No almacena toda la secuencia.                                                                                             |
| Mayor consumo de memoria.                                            | Muy bajo consumo de memoria.                                                                                               |
| Acceso por índice ( <code>lista[500]</code> ).                       | No permite acceso por índice.                                                                                              |
| Puede recorrerse muchas veces.                                       | Normalmente se recorre una sola vez.                                                                                       |
| Ideal para colecciones pequeñas o cuando necesitas acceso aleatorio. | Ideal para archivos grandes, millones de registros, streams de datos y secuencias potencialmente infinitas como Fibonacci. |

## Decoradores

**Definición:** Un decorador "envuelve" una función para ejecutar código antes, después o alrededor de ella.

Es una función que recibe otra función como argumento, le añade un comportamiento adicional y devuelve una nueva función, sin modificar el código original varias funciones:



## Decoradores

### ¿Para qué sirven?

Los decoradores sirven para agregar funcionalidades comunes a muchas funciones, por ejemplo:

- medir tiempos de ejecución;
- registrar eventos (logs);
- validar permisos;
- verificar autenticación;
- controlar acceso;
- repetir ejecuciones;
- capturar excepciones;
- cachear resultados;
- contar llamadas.

Sin decoradores habría que repetir ese mismo código en muchas funciones.

## Decoradores

### ¿Para qué sirven?

Los decoradores sirven para agregar funcionalidades comunes a muchas funciones, por ejemplo:

- medir tiempos de ejecución;
- registrar eventos (logs);
- validar permisos;
- verificar autenticación;
- controlar acceso;
- repetir ejecuciones;
- capturar excepciones;
- cachear resultados;
- contar llamadas.

Sin decoradores habría que repetir ese mismo código en muchas funciones.

## Ejemplo de Decorador

```
def decorador(funcion):  
    def envoltura():  
        print("Inicio")  
        funcion()  
        print("Fin")  
    return envoltura
```

```
@decorador  
def saludar():  
    print("Hola")
```

```
saludar()
```

## Decoradores

### Problema

Tenemos varias funciones:

```
def suma():  
    print("Ejecutando...")  
    return 5 + 3  
  
def resta():  
    print("Ejecutando...")  
    return 8 - 2
```

- Se repite el código, en este ejemplo sencillo el mensaje “Ejecutando”, pero pueden ser varias líneas antes y después de lo que hace realmente cada función.

## Decorador con parámetros

Ejemplo:

```
def decorador(funcion):  
  
    def envoltura(*args, **kwargs):  
  
        print("Ejecutando")  
  
        resultado = funcion(*args, **kwargs)  
  
        return resultado  
  
    return envoltura
```

## Caso Real: medir tiempo

```
import time

def medir_tiempo(funcion):

    def wrapper(*args, **kwargs):

        inicio = time.time()

        resultado = funcion(*args, **kwargs)

        fin = time.time()

        print(fin - inicio)

        return resultado

    return wrapper
```

## Actividad 3

Crear un decorador que:

- Muestre fecha y hora.
- Indique qué función se ejecutó.
- Muestre el resultado.

Ejemplo:

```
[2026-01-10 15:30]
```

```
Función: calcular()
```

```
Resultado: 150
```

## Modularización y Paquetes

### Problema:

- Archivo de 1000 líneas:

```
main.py
```

- Difícil de mantener.

### Solución:

- Separar responsabilidades.

### Estructura:

```
proyecto/  
  
|  
├─ main.py  
├─ calculos.py  
├─ reportes.py  
└─ utilidades.py
```

## Modularización y Paquetes

### Módulos

calculos.py

```
def sumar(a,b):  
    return a+b
```

main.py

```
from calculos import sumar  
  
print(sumar(5,3))
```

### Paquetes

empresa/

```
|  
├── __init__.py  
├── empleados.py  
└── ventas.py
```

## Buenas Prácticas y PEP 8

- **PEP: Python Enhancement Proposal**
- Es la guía oficial de estilo para escribir código Python.
- No cambia cómo funciona el programa, sino cómo debe escribirse para que sea más claro y uniforme.
- Documentación oficial:
  - <https://peps.python.org/pep-0008/>

## Buenas Prácticas y PEP 8

- **Nombres significativos para: variables, funciones, clases**
- **Usar espacios alrededor de operadores, luego de comas**

### Ejemplo incorrecto

```
</> Python
def calc(x,y):return x+y
```

### Nombres descriptivos

```
</> Python
precio_total
```

### Mejor práctica

```
</> Python
def calcular_suma(x, y):
    return x + y
```

### No:

```
</> Python
pt
```

## Buenas Prácticas y PEP 8

### - Longitud de línea

- PEP 8 recomienda máximo 79 caracteres por línea

### Incorrecto

```
print("Esta línea extremadamente larga continúa durante muchos caracteres haciendo difícil leer el programa")
```

### Correcto

```
mensaje = (  
    "Esta línea es mucho más fácil "  
    "de leer."  
)  
  
print(mensaje)
```

## Buenas Prácticas y PEP 8

### - Sangría (Indentación)

- Python utiliza 4 espacios

### **Incorrecto**

```
if edad >= 18:  
    print("Mayor")
```

```
if edad >=18:  
    print("Mayor")
```

### **Correcto**

```
if edad >= 18:  
    print("Mayor de edad")
```

## Buenas Prácticas y PEP 8

### - Líneas en Blanco

- Separar funciones mediante dos líneas, hace el código más legible

```
def suma():  
    pass
```

```
def resta():  
    pass
```

## Buenas Prácticas y PEP 8

### - Comentarios útiles

- Los comentarios deben explicar el porqué, no el qué

```
# Incrementa el número de intentos permitidos  
intentos += 1
```

## Buenas Prácticas y PEP 8

### - Evitar código duplicado

#### - Incorrecto

```
print("Bienvenido Juan")  
  
print("Bienvenido Ana")  
  
print("Bienvenido Luis")
```

#### - Correcto

```
def saludar(nombre):  
    print(f"Bienvenido {nombre}")
```

## Buenas Prácticas y PEP 8

- **Utilizar constantes**

- En Python se escriben en mayúsculas

```
PI = 3.1416
```

```
IVA = 0.15
```

- Aunque Python no impide modificarlas, es una convención

## Buenas Prácticas y PEP 8

### - Evitar variables globales

#### - Incorrecto

```
contador = 0

def incrementar():
    global contador
    contador += 1
```

#### - Mejor

```
def incrementar(contador):
    return contador + 1
```

## Buenas Prácticas y PEP 8

### - Utilizar f-strings

- Incorrecto

```
print("Hola " + nombre)
```

- Mejor

```
print(f"Hola {nombre}")
```

## Buenas Prácticas y PEP 8

- **Escribir funciones pequeñas.** Cada función debe realizar una sola tarea

- Mal

```
def sistema():
```

... 100 líneas

- Mejor

```
leer_datos()
```

```
procesar()
```

```
guardar()
```

```
mostrar()
```

### Mantener funciones pequeñas

Ideal:

```
</> Python
```

```
20-30 líneas
```

## Buenas Prácticas y PEP 8

### - Capturar excepciones correctamente.

#### - Mal

Muy general

```
try:  
    ...  
except:  
    print("Error")
```

#### - Mejor

```
try:  
    numero = int(input())  
except ValueError:  
    print("Debe ingresar un número")
```

## Buenas Prácticas y PEP 8

- **Eliminar código muerto.** Si no se usa, eliminarlo. El control de versiones Git ya guarda el historial.

- Mal

```
# Código viejo
```

```
#print(total)
```

## Buenas Prácticas y PEP 8

### - Importaciones

- Correcto

```
import math

import pandas as pd

import numpy as np
```

- No usar

```
from math import *
```

- Porque contamina el espacio de nombres

## Buenas Prácticas y PEP 8

### - Orden de Importaciones

- PEP 8 recomienda:

1. Librerías estándar
2. Librerías externas
3. Librerías propias

```
import os
import sys

import numpy as np
import pandas as pd

from proyecto import utilidades
```

## Buenas Prácticas y PEP 8

### - Documentar funciones (Docstrings)

- Los docstrings permiten que otros desarrolladores entiendan rápidamente qué hace una función y cómo utilizarla.

```
def calcular_promedio(lista):  
    """  
    Calcula el promedio de una lista de números.  
  
    Parámetros:  
        lista (list): Lista de números.  
  
    Retorna:  
        float: Promedio.  
    """
```

## Herramientas que verifican PEP 8

- En Visual Studio Code, estas herramientas pueden integrarse para mostrar advertencias mientras se programa.
- **Black**: formateador automático.
- **Flake8**: detecta errores de estilo y posibles problemas.
- **Pylint**: evalúa calidad del código y sugiere mejoras.
- **autopep8**: corrige automáticamente muchas reglas de PEP 8.
- **Ruff**: analizador muy rápido que combina varias verificaciones.

## Taller Integrador

### Sistema de Procesamiento de Ventas

#### Datos:

```
ventas = [  
    150,  
    300,  
    450,  
    100,  
    250,  
    600  
]
```

#### Requisitos:

**Parte 1:** Crear un generador que entregue ventas una por una.

**Parte 2:** Crear decorador que registre la ejecución

```
@registrar
```

#### Parte 3: Crear módulo:

```
ventas.py
```

con funciones:

```
calcular_total()  
  
calcular_promedio()  
  
venta_maxima()
```

**Parte 4:** Crear archivo que utilice todo el sistema:

```
main.py
```

## Tarea Autónoma

**Proyecto:** Simulador de Sensores IoT

### Objetivo:

- Simular lecturas de sensores utilizando generadores.

#### Requisitos

1. Crear generador infinito:

```
Python  
sensor_temperatura()
```

2. Generar valores aleatorios:

```
Python  
  
25.1  
24.8  
25.3  
...
```

3. Crear decorador que registre:

- Fecha
- Hora
- Lectura

4. Organizar en módulos:

```
sensores/
```

5. Aplicar PEP 8.

# GRACIAS